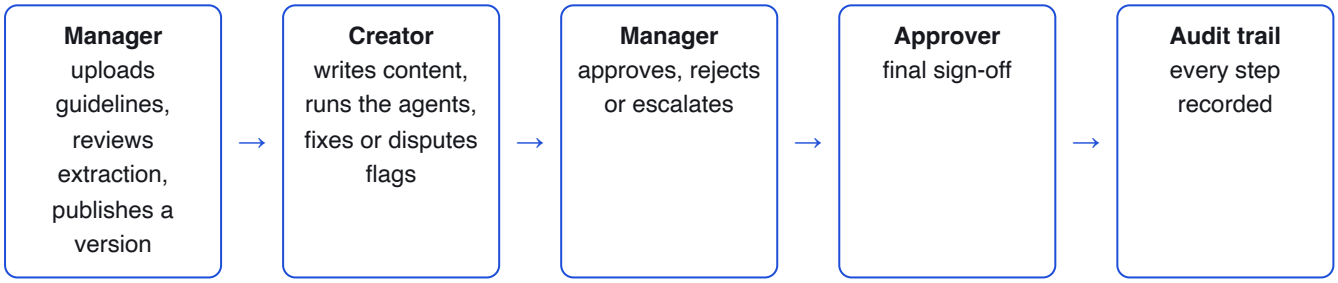
Verity

Intelligent brand governance. Every word. Every pixel. On brand. · Team Merge Conflicts · Inkloom AI Hackathon, Brand Governance track · Round 2 updated project document

Try it (no signup): https://verity-two-delta.vercel.app → click **Start judge walkthrough**. A guide bar at the bottom walks through five steps in about five minutes.
Roles to try: Maya (Creator), Rajan (Manager), Anita (Approver), each one click on the landing page.
Source code (public): https://github.com/Swas00/verity
Note: the site runs on free hosting. If nobody used it for about 15 minutes, the first visit can take 30 to 60 seconds; the landing page says "Waking up the server" and signs you in by itself once ready.

1. What Verity does

Verity checks marketing content against a brand's guidelines and explains every flag with the clause it breaks, a confidence score and a suggested fix. People make the final call: a Creator, Manager, Approver workflow with a tamper-evident audit trail. Nothing is published automatically.



2. Finished and working on the live link

Area	What works
Guideline builder	Upload PDF, DOCX, TXT or MD (or use the sample). Extracts rules, approved terms, approved claims with source pages and a voice profile. Editable preview with notes on anything unclear. Publishing creates a new immutable version.
Review editor	Highlight on every flagged span, severity by colour and underline style, flags follow the text as it is edited.
Agents	**TermGuard** (banned and approved terms, singular, plural and hyphen variants, allowed contexts). **ClaimGuard** (unsupported claims matched to the approved claims register with document and page, wrong numbers flagged). **ToneGuard** (labelled basic mode, see section 5). **MetaAgent** merges overlaps, ranks flags and computes a 0 to 100 score.
Findings panel	Clause citation, quoted text, confidence bar, plain-English reason, suggested fix. Accept fix, edit, dispute (a written reason is required).
Reliability	Results stream as each agent finishes. One failing agent never blocks the others. Result cache, rate limiting, labelled fallback result for the sample text if the AI service is down.
Workflow	Submit, approve, reject with reason, escalate, final approval, send back, publish. Roles enforced on the server. Nobody can approve their own content.

	Queue with score, open flags, change since last revision and age. Comments and in-app notifications.
Revisions and audit	Every run is an immutable revision with a word-level diff and compliance change. Audit log is append-only and hash-chained; "Verify chain" recomputes it; filter and export as CSV or JSON.
Dashboard	Reviews, average score, time to approve, AI versus human flags, score trend, violations by agent, content type and creator, clauses reviewers keep overriding.
Learning loop	A rule that reviewers dispute or override in at least half of 5 or more decisions gets a higher confidence cutoff; borderline flags become suggestions and stop lowering the score.
Engineering	OpenAPI 3.1 reference for all 25 routes (checked against the code by a test), pagination, request ids and logging, security headers, input validation, optional BullMQ and Redis queue, server-waking message.
Accessibility	Automated audit with no serious or critical violations in light and dark mode, keyboard-only journey, no horizontal scroll at 320 px or 200% zoom.

3. Architecture and technology

Service	Technology and role
Web (Vercel)	Next.js 14, React 18, TipTap/ProseMirror. Interface only; every permission is re-checked by the API.
API (Render, free)	Node.js and Express. JWT demo logins, role checks, workflow state machine, SSE streaming, cache, rate limiter, audit log, analytics, guideline versioning.
AI service	Python FastAPI and Pydantic. Runs the agents in parallel and merges them. Same container as the API on Render.
Database (Neon, free)	PostgreSQL with JSONB. `brand_id` on every table. Locally and in tests an in-process Postgres (PGlite).
Queue (optional)	BullMQ and Redis run one job per agent when `REDIS_URL` is set; otherwise a direct pipeline runs. Not enabled on the live demo.

Technically hard parts

- **Span integrity.** Models are poor at character offsets. Agents return the quoted text; the server computes and verifies the offsets, re-locates or drops mismatches; a mapping layer converts text offsets to editor positions. Tested with Unicode, repeated phrases and newlines.
- **Tamper-evident audit log.** One SQL statement computes each hash from the previous row; a unique constraint stops forks under concurrency; triggers reject update, delete and truncate.
- **Calibration from reviewer overrides,** per rule, with the effect visible on the dashboard.
- **Graceful degradation.** Per-agent isolation, a circuit breaker around the optional queue, honest labels on every fallback.

Measured results

Agent	Dev set (precision / recall)	Held-out set (precision / recall)
TermGuard	1.00 / 1.00	0.83 / 1.00

ClaimGuard	0.90 / 1.00	0.75 / 1.00
ToneGuard (basic mode)	1.00 / 1.00	0.79 / 1.00

Two annotated sets of 30 texts each, written by the team. The held-out set was not used to tune any rule and its labels were not changed to improve the numbers. Database indexes were measured on 20,000 generated reviews: first page of a review list 14 ms to 0.07 ms; one review's audit trail 6.7 ms to 0.01 ms. Tests: 48 API, 62 AI, 7 web unit and four browser suites, run by GitHub Actions on every push.

4. What changed from the Round 1 proposal, and why

Round 1	Round 2	Why
MongoDB	PostgreSQL JSONB	One database fewer; findings are JSON anyway.
Pinecone	Lexical retrieval (BM25, weighted coverage)	Character n-gram vectors did worse on our data, and the claims register is small.
Clerk	Own JWT with seeded demo users	Judges try every role in one click, no signup.
Socket.IO	Server-sent events	Simpler, enough for one-way streaming.
BullMQ and Redis	Optional, direct pipeline by default	The live demo works without Redis.
Claude for ToneGuard	Rule-based basic mode live; LLM path built	Zero cost, cannot hit rate limits.
AWS S3, Liveblocks	S3 deferred to Round 3; Liveblocks dropped	Images need VisualGuard; Liveblocks is outside the core workflow.
LLM extraction	Rule-based extraction	Free and deterministic.

No new product features were added beyond the Round 1 set; the additions are engineering quality (API reference, pagination, indexes, logging, accessibility, the waking message).

5. What does NOT work yet

- ToneGuard on the live demo is rule-based basic mode, not the LLM: it catches jargon, hype, shouting and pressure wording, not subtle voice problems. LLM latency has not been measured.
- Free hosting sleeps when idle; the first visit can take 30 to 60 seconds (the page says so).
- Demo data is shared: every visitor sees the same reviews and each other's changes.
- Known detection errors: ClaimGuard flags "only" in "Limited time only" and "best" in "best-in-class"; TermGuard flags "Java" meaning the language and "Joe's Diner" unless the brand lists them as allowed contexts. The test sets are small.
- Guideline extraction is rule-based and can miss unusual layouts; scanned (image-only) PDFs are not read.
- Email notifications are built but off; in-app notifications work.
- Cache and rate limiter are per server process. Sign-in is demo logins only.
- Dashboard summary queries read every review (about 50 to 90 ms at 20,000 reviews); not optimised further.

- No usability test with outside people and no screen-reader testing yet; automated accessibility checks and a keyboard-only run were done.
- Round 3 items not built: VisualGuard, multi-brand workspaces, Brand Drift Intelligence, tone embeddings.

6. Three improvements for Round 3

Improvement	What it changes for the user, and how it will be tested
1. VisualGuard image and logo compliance	A creator uploads a banner and gets the same kind of flag on the image, with the clause, before a manager sees it. Image embeddings against approved assets, palette check, logo clear-space and misuse checks, using the existing finding format with an image region. **Test:** at least 85% recall on a 30-image set, at most 10% false flags on compliant images. **In place:** `brand_assets` table, guarded upload route, `region` field.
2. Multi-brand workspaces agency mode	A manager switches between client brands, each with its own guidelines, reviews and analytics, and shares a read-only compliance report. Per-brand membership, isolation, a cross-brand contamination detector. **Test:** automated tests prove no cross-brand reads or writes; at least 80% of planted cross-brand samples flagged. **In place:** `brand_id` on every table, scoped queries, a second brand in the seed.
3. Brand Drift Intelligence	A brand manager sees which teams and clauses drift over time and is alerted before violations pile up. Rolling-window tone analysis against the brand centroid, per-team drift score, weekly digest. **Test:** on a seeded history with planted drift, the alert fires within one window and not on a stable control series. **In place:** stored revision history, 40 seeded reviews, override-rate analysis.

7. Team and disclosures

Name	Round 1 role	Round 2 contribution (as shown by the repository)
Swastik Sahu	Full-Stack Lead and AI/ML Engineer	Project lead and integration: web app, API, AI agents and evaluation, deployment, CI, documentation, review of pull requests.
Akriti Tigga	Backend Engineer	
Gayathri VS	Frontend Engineer and UI/UX Designer	Accessible reason dialog (focus, keyboard, validation) with end-to-end test updates.
Sakshi Kumari	AI / ML Engineer	
Shivam Agarwal	Backend Engineer and DevOps	Optional BullMQ and Redis queue, single-agent and merge endpoints, queue tests and benchmark, CI jobs with and without Redis, documentation.

AI assistance: the team used Claude (Anthropic) while building, including Claude Code for writing and testing parts of the code, tests and documentation; the team directed and reviewed the work. **Licence:** MIT. **Demo data:** "Lumina Coffee Co." and all reviews, users and claims are fictional.